

Tema 4: NP-completitud (2ª Parte)

Serafín Moral

Universidad de Granada

Abril, 2024

Técnicas de Reducción

Del libro de Garey-Johnson:

- **Restricción.** Demostrando que un problema NP-completo es un subproblema del que estamos considerando.
- **Reemplazamiento Local.** Haciendo una transformación de un problema NP-completo elemento a elemento.
 - **Reemplazamiento Local con Refuerzo.** Haciendo una transformación de un problema NP-completo elemento a elemento, pero añadiendo algunos elementos adicionales para forzar la equivalencia de los problemas.
- **Diseño de Componentes.** Distintos elementos del problema original se transforman en distintos tipos de estructura que se conectan con otros elementos.

Regla

Si un problema Π_1 es NP y un subproblema suyo, Π_2 , es NP-completo, entonces Π_1 es NP-completo.

'Subproblema' significa que eligiendo unos parámetros concretos de Π_2 , obtenemos Π_1 .

Los padres heredan la dificultad de los hijos. Si Π_2 es NP-difícil $\Rightarrow$ el padre lo es.

Si $\Pi_2 \in \text{NP} \Rightarrow \Pi_1$ estará, mínimo, en NP.

Realmente, $\Pi_2 \neq \Pi_1$.

Ejemplo: Problema de la Mochila

Tenemos un conjunto finito de objetos U . Cada objeto, u , tiene un tamaño, $s(u) \in \mathbb{N}$, y un valor $v(u) \in \mathbb{N}$. Tenemos, además, dos números naturales: B (el tamaño máximo) y K (el valor mínimo). La pregunta es si existe un subconjunto de objetos $U' \subseteq U$, tal que

$$\sum_{u \in U'} s(u) \leq B, \quad \sum_{u \in U'} v(u) \geq K$$

El problema de la partición es un caso particular de este problema, en el que $s(u) = v(u), \forall u$ y $B = K = (1/2) \sum_{u \in U} s(u)$.

Ejemplo: Problema de la Mochila

Tenemos un conjunto finito de objetos U . Cada objeto, u , tiene un tamaño, $s(u) \in \mathbb{N}$, y un valor $v(u) \in \mathbb{N}$. Tenemos, además, dos números naturales: B (el tamaño máximo) y K (el valor mínimo). La pregunta es si existe un subconjunto de objetos $U' \subseteq U$, tal que

$$\sum_{u \in U'} s(u) \leq B, \quad \sum_{u \in U'} v(u) \geq K$$

El problema de la partición es un caso particular de este problema, en el que $s(u) = v(u), \forall u$ y $B = K = (1/2) \sum_{u \in U} s(u)$.

REDUCCIÓN de PARTICIÓN(C,s) a MOCHILA(U,s',v',B,K):
 $U = C, s' = s, v' = s, B = K = (1/2) \sum_{u \in C} s(u)$

Asignación en un Multiprocesador

Datos: Un conjunto A de tareas, cada tarea, $a \in A$, tiene una longitud $l(a) \in \mathbb{N}$. Tenemos, además, un número de procesadores m y un tiempo límite, $D \in \mathbb{N}$.

Pregunta: ¿Existe una partición de A : $\{A_1, \dots, A_m\}$ en m subconjuntos disjuntos, de manera que

$$\max \left\{ \sum_{a \in A_i} l(a) : 1 \leq i \leq m \right\} \leq D$$

Si nos restringimos al caso $m = 2$ y $D = 1/2 \sum_{a \in A} l(a)$ obtenemos el problema de la partición.

Reemplazamiento Local

Cada elemento de un problema NP-completo Π' se transforma en una estructura del problema que estamos considerando Π .

Ejemplo: Reducción de SAT a 3-SAT

Partición en Triángulos (PARTRI)

Se parte de un grafo $G = (V, E)$ con un número de vértices $|V| = 3q$.

La pregunta es si existe una partición de V en q conjuntos disjuntos de tamaño 3: $V_1, \dots, V_q$, de tal manera que si $V_i = \{V_{i[1]}, V_{i[2]}, V_{i[3]}\}$, entonces los arcos $(V_{i[1]}, V_{i[2]}), (V_{i[2]}, V_{i[3]}), (V_{i[1]}, V_{i[3]}) \in E$ (los subconjuntos de vértices son triángulos).

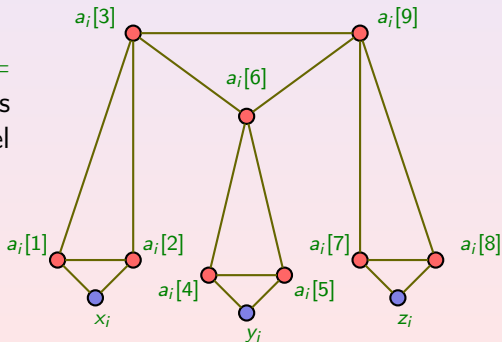
Este es un problema de NP, y se puede demostrar que es completo reduciendo 3-SET (cobrimiento por conjuntos de tamaño 3).

Reducción: 3-SET $\propto$ PARTI

Sea X con $|X| = 3q$ y C una familia de subconjuntos de tres elementos de X .

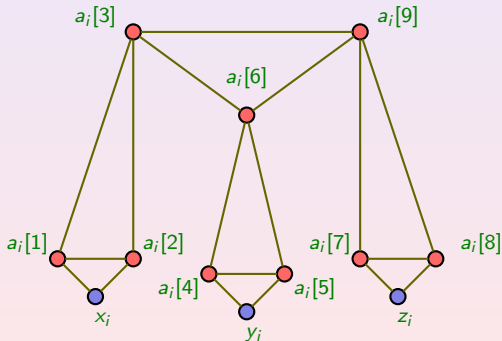
Construimos el grafo $G = (V, E)$ con un número de vértices $|V| = 3q'$ de la siguiente forma: todos los elementos de X serán vértices del grafo.

Para cada conjunto $c_i = \{x_i, y_i, z_i\} \in C$, añadimos 9 nuevos vértices a V y el siguiente subgrafo:



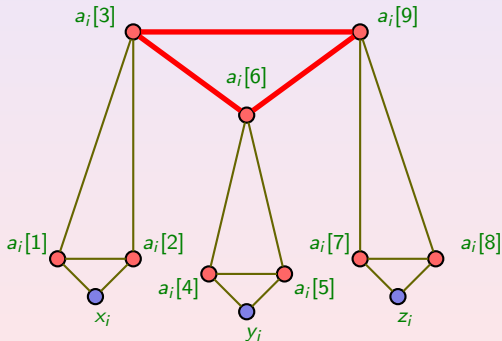
Equivalencia de los Problemas

La equivalencia de las soluciones en ambos problemas se obtiene de las dos formas que existen de elegir los triángulos en cada uno de los subgrafos de G :



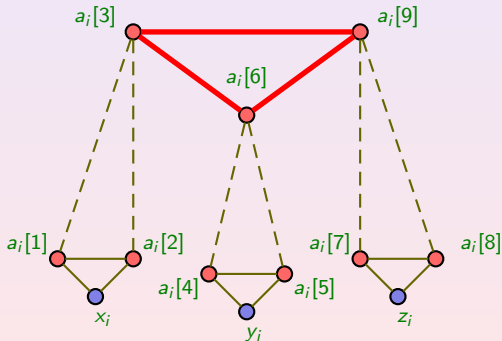
Equivalencia de los Problemas

La equivalencia de las soluciones en ambos problemas se obtiene de las dos formas que existen de elegir los triángulos en cada uno de los subgrafos de G :



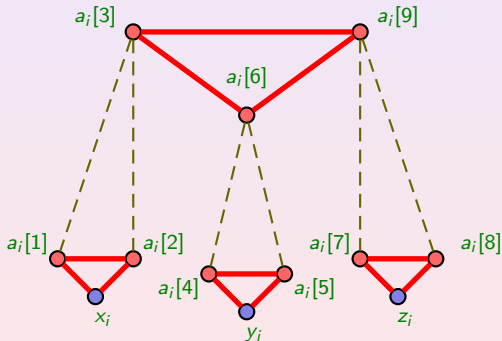
Equivalencia de los Problemas

La equivalencia de las soluciones en ambos problemas se obtiene de las dos formas que existen de elegir los triángulos en cada uno de los subgrafos de G :



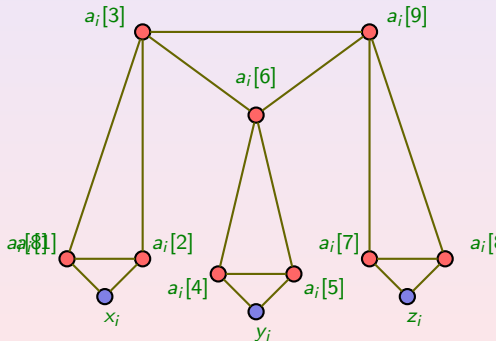
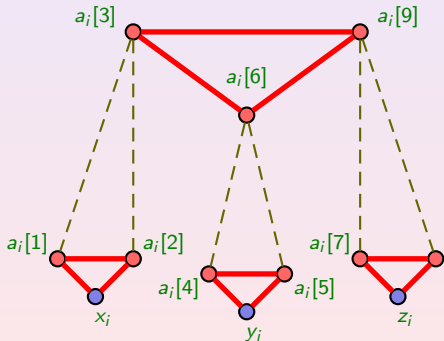
Equivalencia de los Problemas

La equivalencia de las soluciones en ambos problemas se obtiene de las dos formas que existen de elegir los triángulos en cada uno de los subgrafos de G :



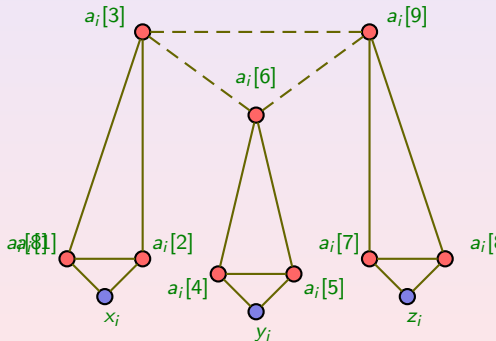
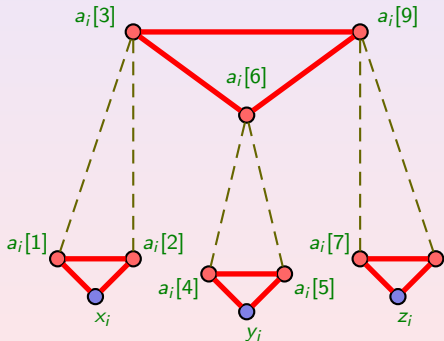
Equivalencia de los Problemas

La equivalencia de las soluciones en ambos problemas se obtiene de las dos formas que existen de elegir los triángulos en cada uno de los subgrafos de G :



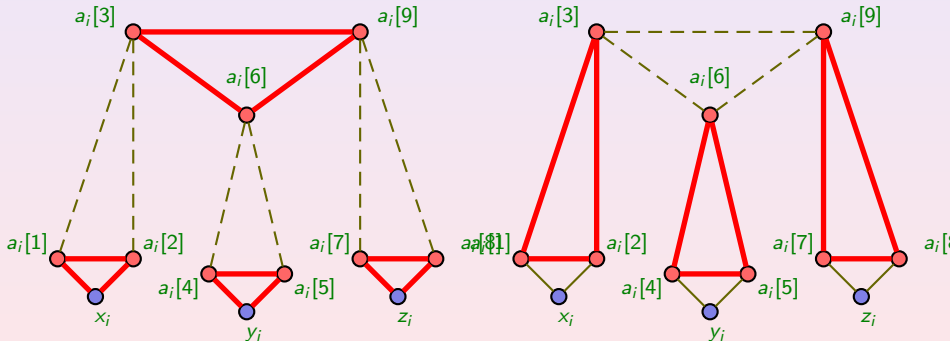
Equivalencia de los Problemas

La equivalencia de las soluciones en ambos problemas se obtiene de las dos formas que existen de elegir los triángulos en cada uno de los subgrafos de G :



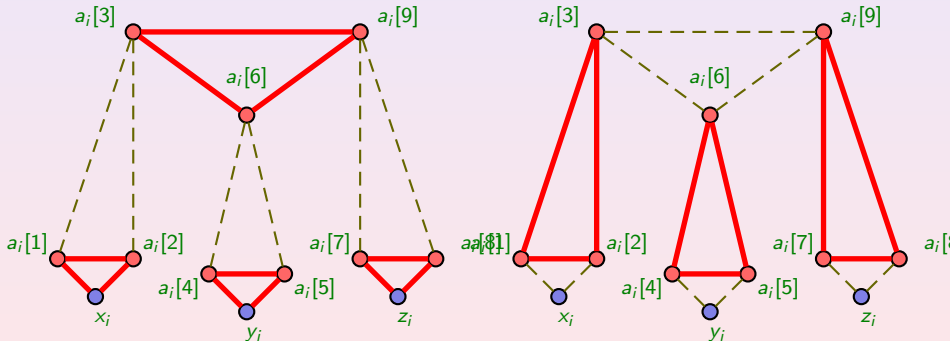
Equivalencia de los Problemas

La equivalencia de las soluciones en ambos problemas se obtiene de las dos formas que existen de elegir los triángulos en cada uno de los subgrafos de G :



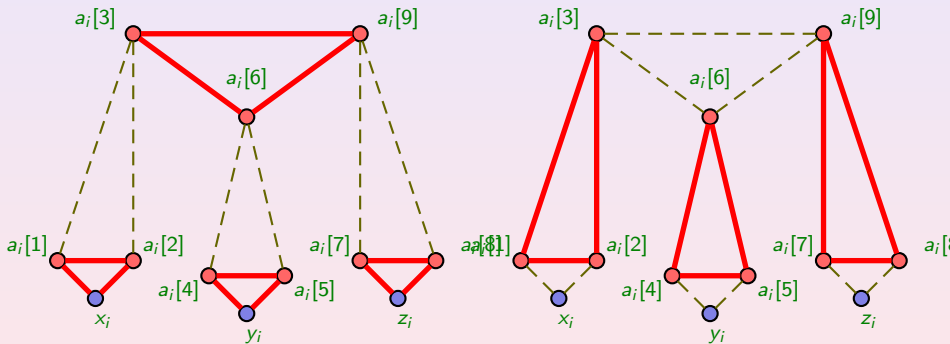
Equivalencia de los Problemas

La equivalencia de las soluciones en ambos problemas se obtiene de las dos formas que existen de elegir los triángulos en cada uno de los subgrafos de G :



Equivalencia de los Problemas

La equivalencia de las soluciones en ambos problemas se obtiene de las dos formas que existen de elegir los triángulos en cada uno de los subgrafos de G :



La primera corresponde al caso en que $\{x_i, y_i, z_i\}$ está en el cubrimiento C' , la segunda al caso en el que no está.

Reemplazamiento Local con Refuerzo

Corresponde al caso en el que, además de los elementos en que se transforman los elementos de Π' , se añaden algunos elementos adicionales para forzar la equivalencia de las soluciones.

Ejemplo: La reducción del cubrimiento por tripletas al problema de la partición: $\text{ACTRI} \propto \text{PARTICION}$.

Conjunto Mínimo de Tests (CMT)

Datos: Un conjunto finito A (posibles diagnósticos), una familia C de subconjuntos de A (posibles tests) y un entero $J \in \mathbb{N}$ que representa el número admisible de tests.

Pregunta: ¿Existe una subfamilia de test $C' \subseteq C$ con $|C'| \leq J$ y tal que para cada par de elementos distintos $a_i, a_j \in A$, existe un test $c \in C'$ que contiene uno y sólo uno de los elementos del par (un elemento, por ejemplo, a_i está en c y el otro fuera)?

Reducción: ACTRI a CMT

Vamos a reducir ACTRI a este problema. Supongamos un ejemplo de ACTRI con $M \subseteq W \times X \times Y$ y $|W| = |X| = |Y| = q$.

Vamos a crear un ejemplo de este problema equivalente a él.

Hacemos $A = W \cup X \cup Y \cup \{w_0, x_0, y_0\}$

$C = \{\{w, x, y\} : (w, x, y) \in M\} \cup \{W \cup \{w_0\}, X \cup \{x_0\}\}$

$J = q + 2$

Distintas componentes del problema a reducir Π' se transforman en distintas estructuras de Π que se conectan de alguna forma para forzar la equivalencia.

Ejemplos:

- Cubrimiento por vértices
- El circuito Hamiltoniano
- La demostración del teorema de Cook

Reducibilidad Turing

Un problema Π se reduce Turing a Π' lo que se representa como

$$\Pi \propto_T \Pi'$$

si y solo si Π se puede resolver en tiempo polinómico mediante un algoritmo que puede llamar a una función que resuelve Π' contando cada llamada como un paso de cálculo.

La reducibilidad Turing es un concepto *más débil* que el que hemos visto de reducibilidad espacio logarítmica: Si Π se reduce a Π' , entonces se puede construir una reducibilidad Turing.

Problemas NP-Difíciles

Un problema Π es NP-difícil si y solo si existe un problema NP-completo Π' que se puede reducir (Turing) a Π :

$$\Pi' \propto_T \Pi$$

Teorema

Si un problema NP-difícil se resuelve en tiempo polinómico entonces $P = NP$.

En este tema vamos a considerar problemas NP-difíciles que tienen una dificultad *similar* a los NP-completos (existe reducción Turing entre ambos):

- Clase **Co-NP**: Complementarios de los problemas de NP
- Clase **FNP**: Problemas que buscan una solución, cuando saber si existe es NP.

$$\text{CoNP} = \{\bar{L} : L \in \text{NP}\}$$

Ejemplo

Dado un conjunto ϕ de fórmulas en lógica proposicional determinar si son válidas (se satisfacen para todas las asignaciones de valores de verdad).

Ejemplo

Dado un grafo determinar si NO tiene un circuito hamiltoniano.

- Estos problemas no están en NP, lo que tienen es una Máquina polinómica no determinista en la que la respuesta es afirmativa al problema si **TODAS** las opciones responden **SI**: Si la respuesta es **positiva TODAS** las opciones acaba en **SI**; si la respuesta es **negativa AL MENOS UNA** opción acaba en **NO**.
- Tampoco se reducen a los problemas NP con una transformación espacio logarítmica, ya que estas transformaciones exigen que las respuestas a ambos problemas sean **las mismas**, pero **sí** existen reducciones Turing entre los problemas de NP y los de CoNP.

Problemas CoNP Completos

Definición

Un problema es **CoNP Completo** si y solo si está en la clase CoNP y cualquier otro problema de CoNP se reduce a él.

Teorema

L es NP Completo $\Leftrightarrow \bar{L}$ es CoNP Completo

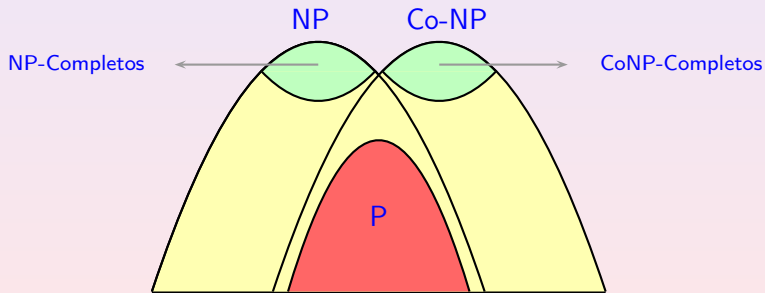
Teorema

Si un problema CoNP completo está en NP, entonces $CoNP = NP$.

Teorema

Si $P = NP$, entonces $CoNP = NP$ (no se sabe $\Leftarrow$)

Estructura de Clases



Caracterizaciones

NP

Un problema $P(x)$ con entrada $x \in A^*$ está en **NP** si y solo si existe una relación R en $A^* \times A^*$ calculable en tiempo polinómico y un polinomio $p(n)$ tal que

$$P(x) = 'Si' \Leftrightarrow \exists y \in A^* \text{ con } |y| \leq p(|x|), \quad R(x, y) = 1$$

Se dice que los problemas de NP son los problemas que se pueden *verificar* en tiempo polinómico (de forma eficiente). Al algoritmo que calcula R se le llama un **verificador**. A y se le llama un **certificado**. → La verificación es determinista

CoNP

Un problema $P(x)$ está en **CoNP** si y solo si existe una relación R en $A^* \times A^*$ calculable en tiempo polinómico y un polinomio $p(n)$ tal que

$$P(x) = 'Si' \Leftrightarrow \forall y \in A^* \text{ con } |y| \leq p(|x|), \quad R(x, y) = 1$$

Primalidad está en NP

Teorema

Un número $p > 1$ es primo si y solo si existe un número $1 < r < p$ tal que $r^{p-1} = 1 \bmod p$ y, además, $r^{\frac{p-1}{q}} \neq 1 \bmod p$, para todos los divisores primos q de $p-1$.

Teorema: Teorema de Pratt

Primos está en NP.

Demostración

Si p es primo, esto se puede certificar con la existencia de un número r tal que $r^{p-1} = 1 \bmod p$ y que además verificase $r^{\frac{p-1}{q}} \neq 1 \bmod p$, para todos los divisores primos q de $p-1$.

La comprobación de que $r^{p-1} = 1 \bmod p$ se puede hacer en tiempo polinómico en función de la longitud de p , que es de orden $l = \log(p)$: Para calcular r^{p-1} , se calculan $r^2, r^4, \dots, r^{2^l}$. Esto tiene un orden de $O(l^3)$.

r por sí solo no es un certificado: $20^{21-1} = 1 \bmod 21$ y 21 no es primo

Demostración

El certificado tiene que adjuntar todos los divisores primos de $p-1$. Es decir constará, en principio de r y una lista de divisores de $p-1 : (q_1, \dots, q_k)$. El comprobar que la lista es completa se hace por divisiones sucesivas de $p-1$ entre estos números y comprobando que de uno al final.

Nos queda una cuestión, ¿cómo sabemos que los números $(q_1, \dots, q_k)$ son primos? Pues dando certificados para ellos. Excepto para 2 que no necesita certificado.

El certificado sería una estructura recursiva: un certificado del número p sería r y una lista de divisores completa de $p-1 : (q_1, \dots, q_k)$, y certificados de primalidad para cada uno de ellos.

Ejemplo

$\text{Cert}(67) = ((67: 2 (2, 3, 11)), (3: 2 (2)), (11: 8 (2,5)), (5: 3 (2)))$

La longitud del certificado es de orden $\log^2(p)$.

La longitud de un número y su lista es de orden $\log(p)$.

El número de listas es de orden $\log(p)$.

Problemas de Búsqueda

Sea A un alfabeto y R una relación en $A^* \times A^*$, el **problema de búsqueda** $P(x)$ asociado a esta relación consiste en dado un ejemplo $x \in A^*$, calcular

$$P(x) = \begin{cases} y & \text{tal que } R(x, y) \text{ si } \{z : R(x, z) = 1\} \neq \emptyset \\ \varepsilon & \text{si } \{z : R(x, z) = 1\} = \emptyset \end{cases}$$

Ejemplo

Dado un conjunto de cláusulas calcular una asignación de valores de verdad, si existe, que satisfaga todas las cláusulas.

Ejemplo

Dado un grafo calcular un circuito hamiltoniano (si este existe)

NP

Un problema $P(x)$ con entrada $x \in A^*$ está en **NP** si y solo si existe una relación R en $A^* \times A^*$ calculable en tiempo polinómico y un polinomio $p(n)$ tal que

$$P(x) = 'Si' \Leftrightarrow \exists y \in A^* \text{ con } |y| \leq p(|x|), \quad R(x, y) = 1$$

Caracterización de FNP

Un problema $P(x)$ está en **FNP** si y solo si está asociado a una relación R decidable en tiempo polinómico y tal que si $R(x, y) = 1$, entonces $|y| \leq p(|x|)$ para un polinomio p :

$$P(x) = \begin{cases} y & \text{tal que } R(x, y) \text{ si } \{z : R(x, z) = 1\} \neq \emptyset \\ \varepsilon & \text{si } \{z : R(x, z) = 1\} = \emptyset \end{cases}$$

La clase FP

FP

FP: La clase de problemas de funciones de *FNP* tales que existe una máquina de Turing determinista que las calcula en tiempo polinómico. *la MT que encuentra las soluciones es determinista.*

FNPT

FNPT: La clase de los problema de FNP totales.
Si para todo x existe un y con $|y| \leq p(|x|)$ tal que $R(x, y) = 1$

Ejemplo

Dado un número p encontrar una descomposición en números primos

$$p = p_1^{k_1} \cdot p_2^{k_2} \cdot \dots \cdot p_m^{k_m}$$

con un certificado de primalidad para cada número primo.
Seguro que existe, pero no es fácil encontrarlo.

Reducciones en FNP

Reducciones en FNP

Un problema de funciones Π se *reduce* a un problema Π' si y solo si, existen funciones R y S calculables en espacio logarítmico, tal que para toda cadena x y z ocurre lo siguiente: Si x es un ejemplo de Π entonces $R(x)$ es un ejemplo de Π' . Además si z es una solución correcta de $R(x)$ entonces $S(z)$ es una solución correcta de x . Es decir,
 $\Pi(x) = S(\Pi'(R(x)))$

Problemas FNP-completos

Con esto podemos definir el concepto de completitud. Un problema es **FNP-completo** si y solo si está en **FNP** y cualquier otro problema de esta clase se puede reducir a este problema.

Ejemplo

Un problema FNP-completo: FSAT

Dado un conjunto de cláusulas, encontrar, si existe, una asignación de valores de verdad para la que todas las cláusulas sean verdaderas.

Teorema

$NP = P$ si y solo si $FNP = FP$

Demostración

La demostración es muy sencilla: si $FNP = FP$ entonces es inmediato que $NP = P$.

Para la implicación inversa, supongamos que $NP = P$, entonces la consistencia se puede resolver en tiempo polinómico.

Ahora, vamos a encontrar una asignación (si existe) en tiempo polinómico. Supongamos que tenemos un conjunto de cláusulas C . Elegimos una variable x_n , y consideramos C_1 que es C suponiendo x_n verdadero (todas las cláusulas con x_n se eliminan y en aquellas en las que aparezca $\neg x_n$ se elimina este literal) y C_2 que es C suponiendo x_n falso.

Demostración

- Si C_1 y C_2 son inconsistentes, C también lo es y no existe la asignación de valores de verdad.
- Si C_1 es consistente, entonces si existe la asignación. Hacemos x_n verdadero y calculamos el valor de verdad de las otras variables, repitiendo de forma recursiva estos pasos sobre C_1 .
- Si C_2 es consistente, entonces si existe la asignación. Hacemos x_n falso y calculamos el valor de verdad de las otras variables, repitiendo de forma recursiva estos pasos sobre C_2 .

Si pudiésemos resolver el problema del viajante en versión de decisión en tiempo polinómico, podríamos resolver el problema de encontrar el circuito óptimo en tiempo polinómico.

- Primero se **obtendría una cota superior R** para el **coste del circuito** óptimo: el valor de distancia más grande, multiplicado por el número de ciudades.
- Después realizando una búsqueda binaria en el intervalo **$[0, R]$** mediante sucesivas llamadas al problema de decisión **se calcula el valor del circuito óptimo: K** .

El Viajante de Comercio (Cont.)

A continuación para cada par de ciudades se hace lo siguiente:
si el coste de ir de una a otra es c , se llama al problema de decisión con presupuesto K e incrementando el coste de c a $c+1$.

- Si la respuesta es **positiva**, **no es necesario usar este arco en el óptimo**. Entonces se deja el coste a $c+1$ y se continúa.
- Si la respuesta es **negativa**, **este arco sí se usa**. Dejamos el coste al mismo valor que estaba antes y continuamos.

Al final obtenemos todos los arcos de un circuito óptimo.

Si en el primer caso se volviera al coste original, en vez de dejarlo en $c+1$, puede que no se obtenga el óptimo, ya que si hay dos circuitos óptimos ninguno de los arcos de ambos circuitos son **necesarios**.

Son los de la clase **FNPT**

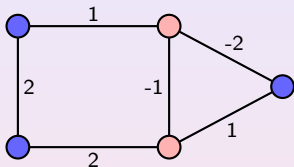
Un problema está en esta clase si para todo x existe un y tal que $R(x, y)$.

Es decir, el problema de decisión no es difícil: siempre tiene una respuesta positiva.

Eso no quiere decir que el problema sea fácil, ya que encontrar la solución no tiene por qué ser inmediato.

Muchos de ellos no se conoce que estén en FP.

Datos: Un grafo (V, E) y un peso w (que puede ser positivo o negativo para cada arco).



Un estado es una aplicación $s: V \rightarrow \{-1, 1\}$ (-1 ●, 1 ●).
El nodo i es feliz si

$$s(i) \cdot \sum_{(i,j) \in E} s(j) \cdot w(i,j) \geq 0$$

Solución: Un estado s en el que todos los nodos sean felices.

El problema está en FNP y es total.

Consideremos

$$\Phi(s) = \sum_{(i,j) \in E} s(i)s(j)w(i,j)$$

Si tenemos un nodo infeliz, i , en s , entonces

$$s(i) \cdot \sum_{(i,j) \in E} s(j) \cdot w(i,j) = -\delta < 0$$

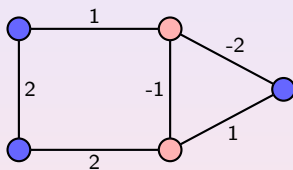
Sea el estado s' igual que s , excepto que $s'(i) = -s(i)$.

Entonces $\Phi(s') = \Phi(s) + 2\delta$.

Como el valor de Φ no puede crecer indefinidamente, este proceso tiene que terminar con una red feliz.

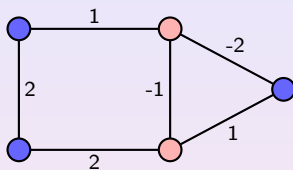
Este algoritmo es pseudo polinómico, pero no polinómico.

Ejemplo



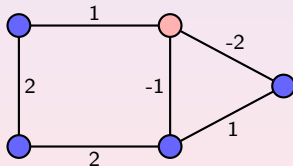
$$\Phi = -1$$

Ejemplo



$$\Phi = -1$$

Cambiamos un nodo infeliz:



$$\Phi = 7$$

La red ya es feliz.

Estructura de Clases de Funciones

Sí sabe que $FP \neq TFNP$, pues hay problemas polinómicos que no tienen sol. garantizada.

No se sabe $TFNP \subseteq FP$. También se sabe $FP \cap TFNP \neq \emptyset$

